

High Performance Computing for Weather and Climate (HPC4WC)

Content: Distributed Memory Parallelism / MPI

Lecturers: Oliver Fuhrer

Block course 701-1270-00L

Summer 2026

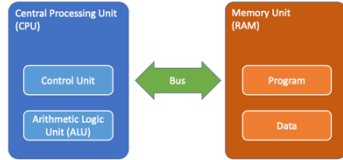


Learning Goals

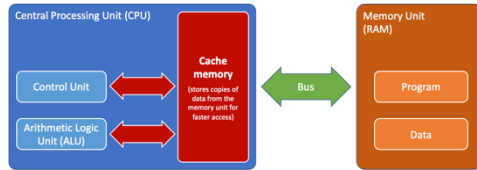
- Understand distributed memory parallelism and how it is different from shared memory parallelism
- Learn basic message passing patterns using MPI
- Be able to apply domain decomposition for solving partial differential equations
- Understand the concept of halo points and able to implement a halo-update.

Computer Architecture

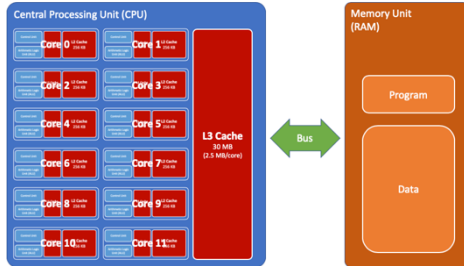
Von Neumann



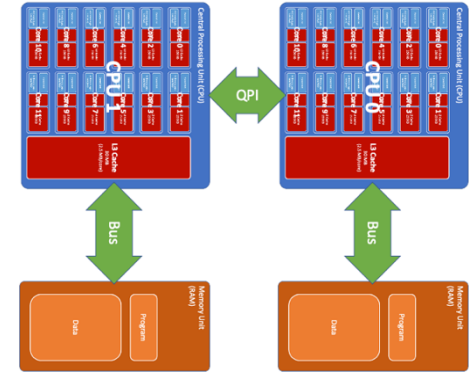
Cache hierarchy



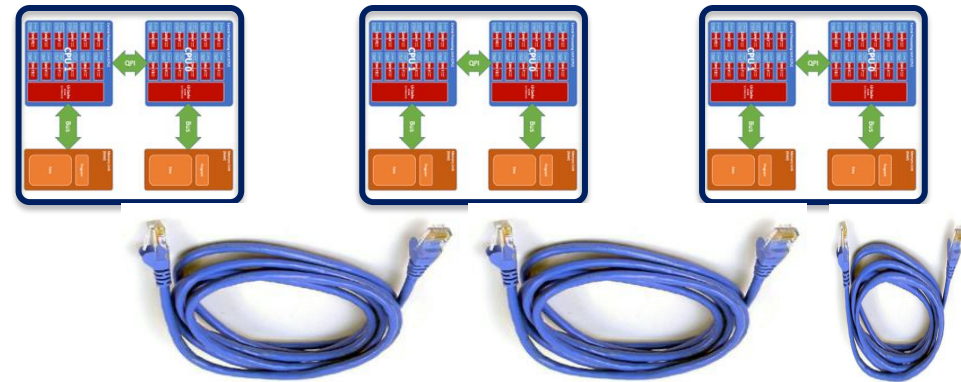
Modern CPU



Multicore Node



Many nodes

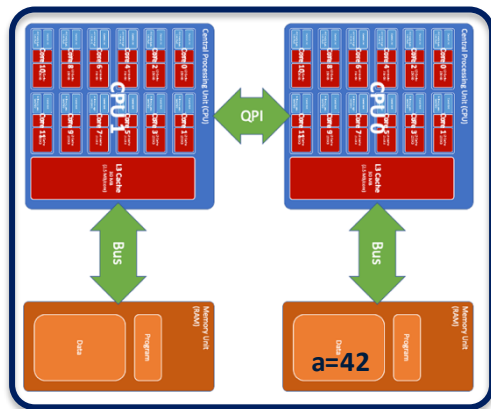


Shared vs. Distributed Memory

All cores on a node share the same address space / memory

```
>>> print(a)
```

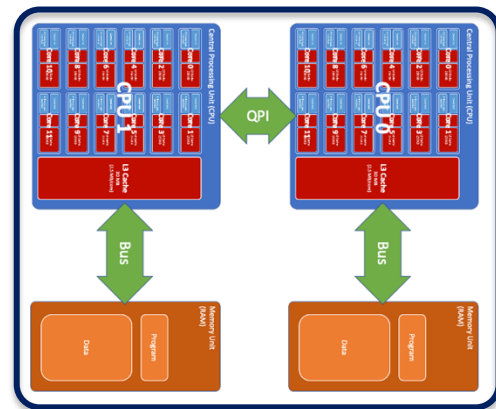
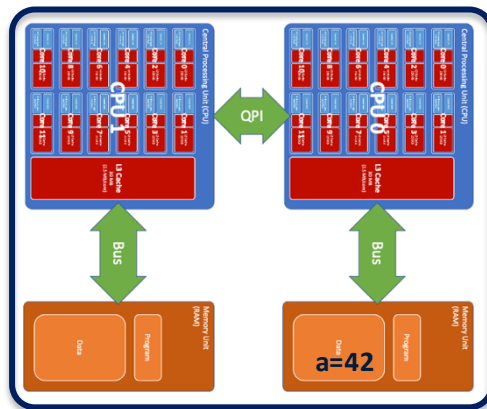
42



Nodes have different address spaces / memories. Variables are not shared.

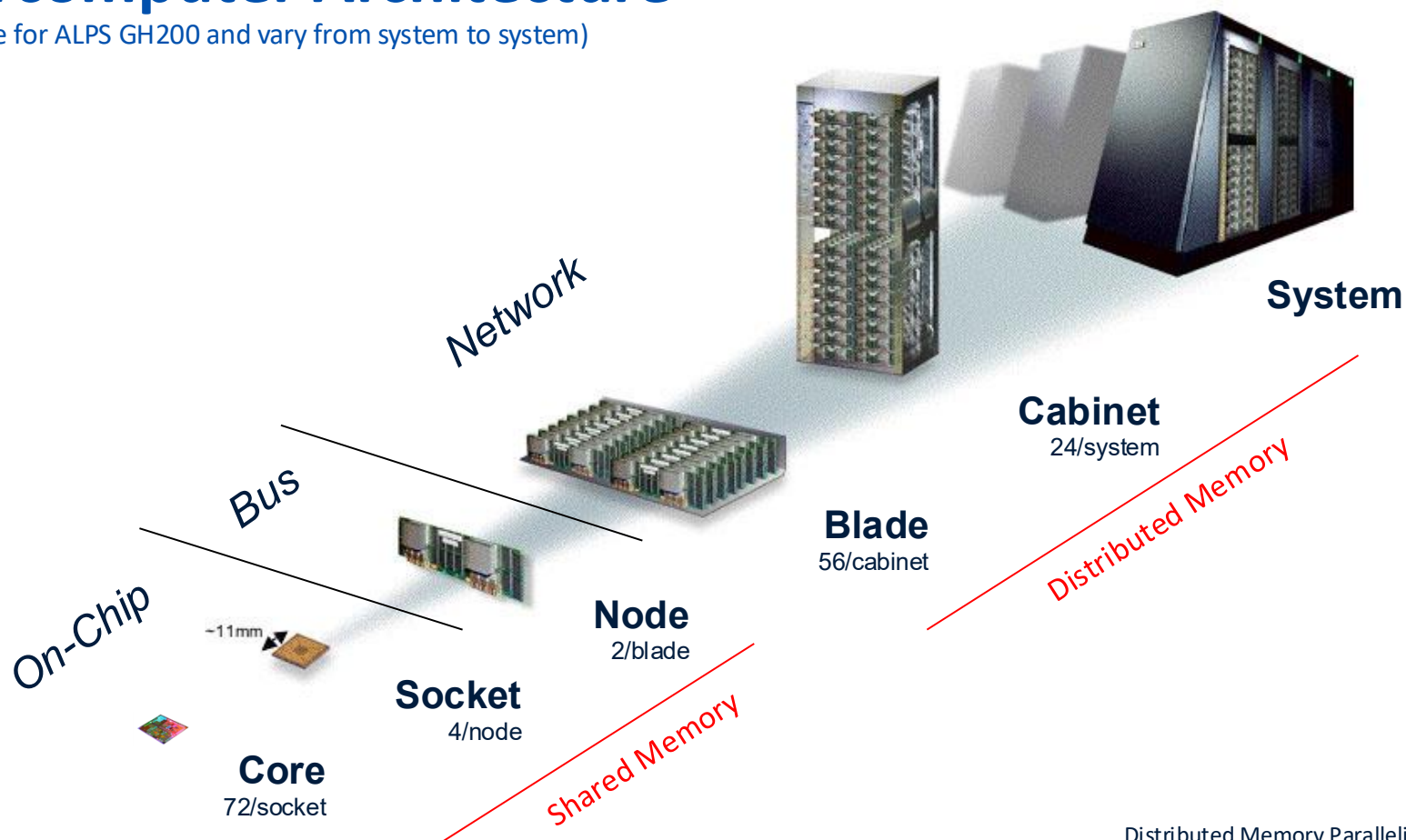
```
>>> print(a)
```

NameError: name 'a' is not defined



Supercomputer Architecture

(Numbers are for ALPS GH200 and vary from system to system)

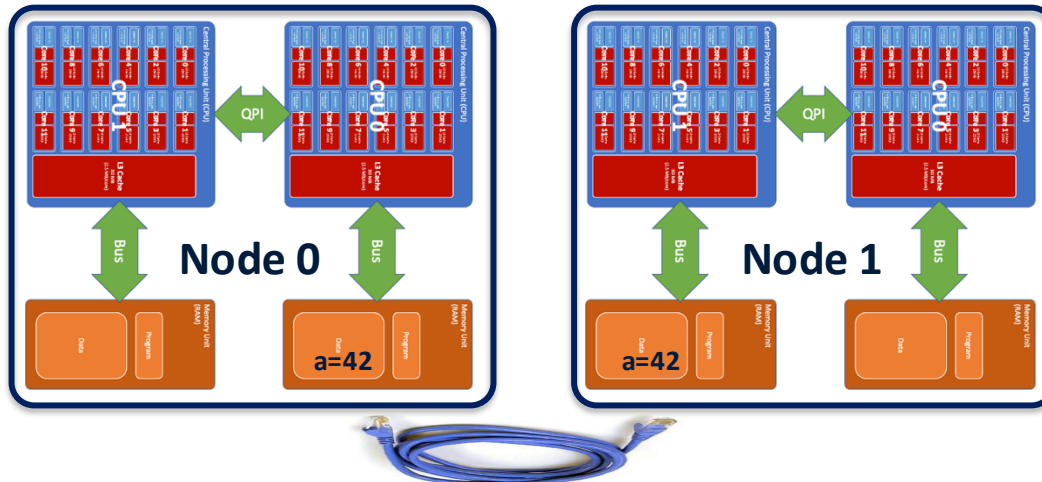


Message Passing

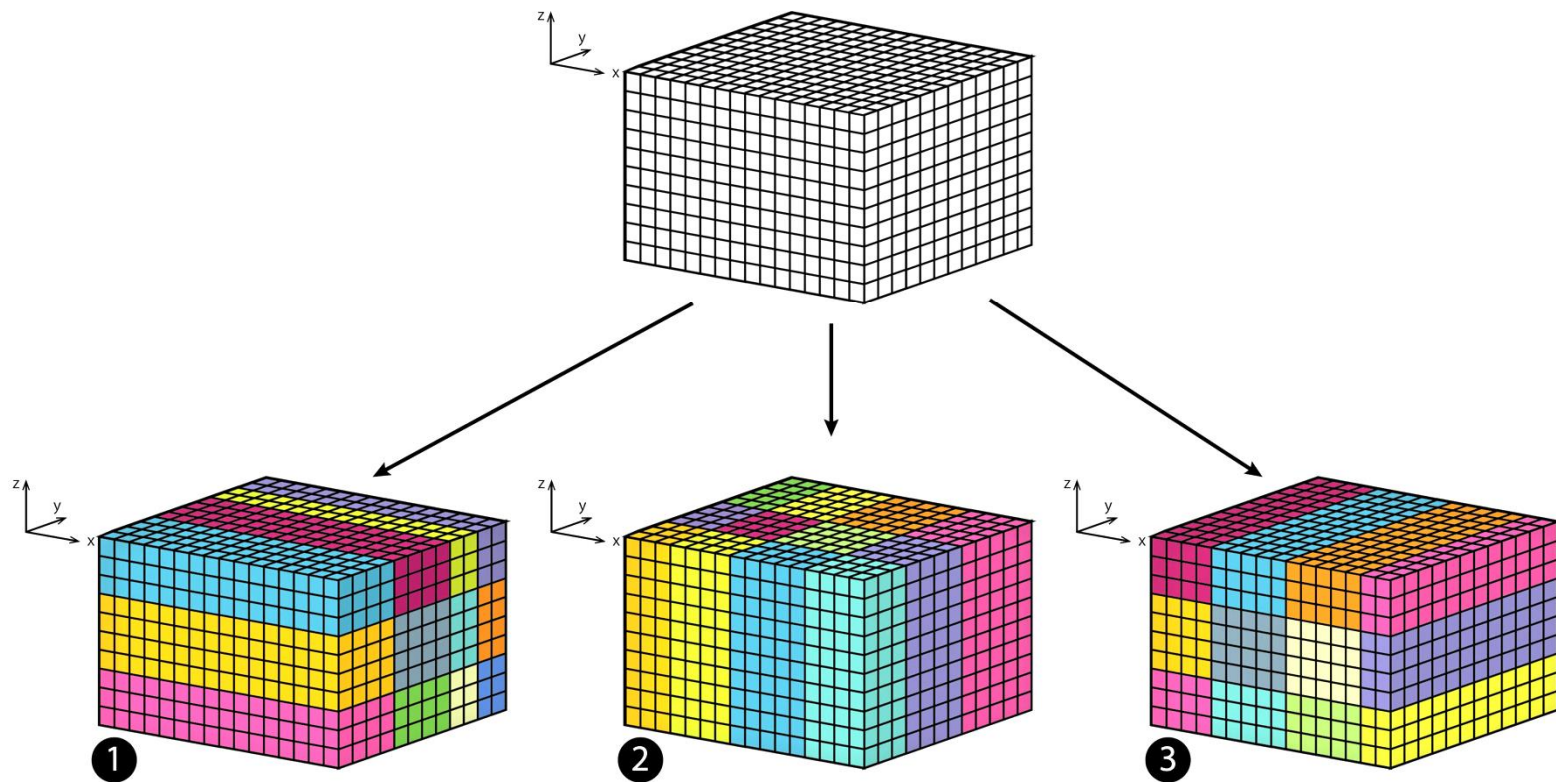
- Information between nodes is transferred over a network cable using a message passing protocol.

```
>>> a = 42  
>>> address(a)  
0x001a947e3211  
>>> send(a, destination=1)
```

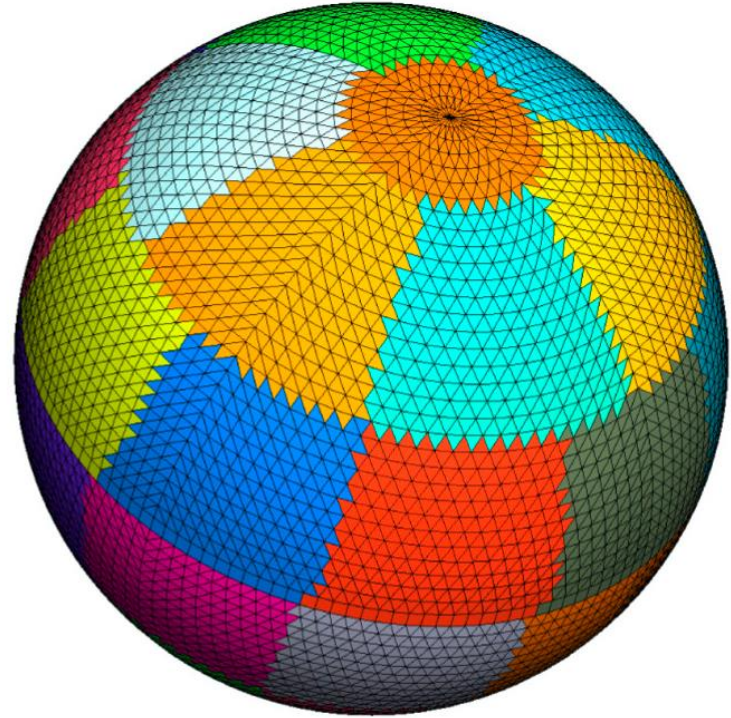
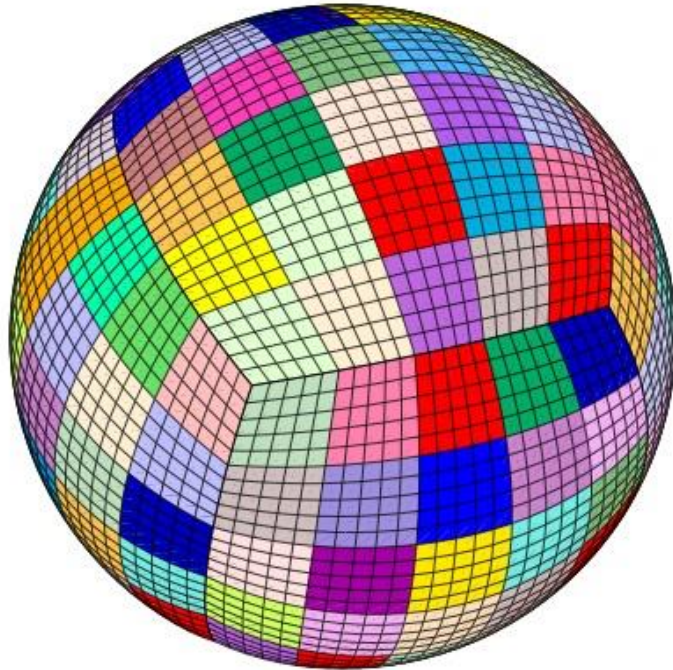
```
>>> a = recv(source=0)  
>>> print(a)  
42  
>>> address(a)  
0x002f33498e77
```



Domain Decomposition



Domain Decomposition in Atmospheric Models



Message Passing Interface (MPI)



- MPI is a standardized and portable message passing standard.
(<https://www.mpi-forum.org/> and <https://github.com/mpi-forum>)
- Version 1.0 in 1992, latest Version 5.0 in June 2025
- Support for Fortran, C, C++, Python, Julia, ...
- Implemented as a library that provides message passing semantics.
- Several implementations
 - MVAPICH
 - OpenMPI
 - Cray MPI
 - ...
- Available on almost any architecture
 - Linux Laptop (apt-get install mpich)
 - Supercomputer
 - Google Cloud Platform
 - ...

Lab Exercises

01-test-MPI-setup.ipynb

- Test the setup of your JupyterHub Server to make sure that MPI is working correctly.

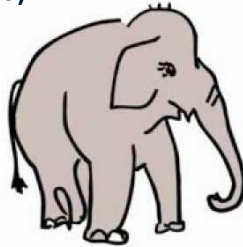
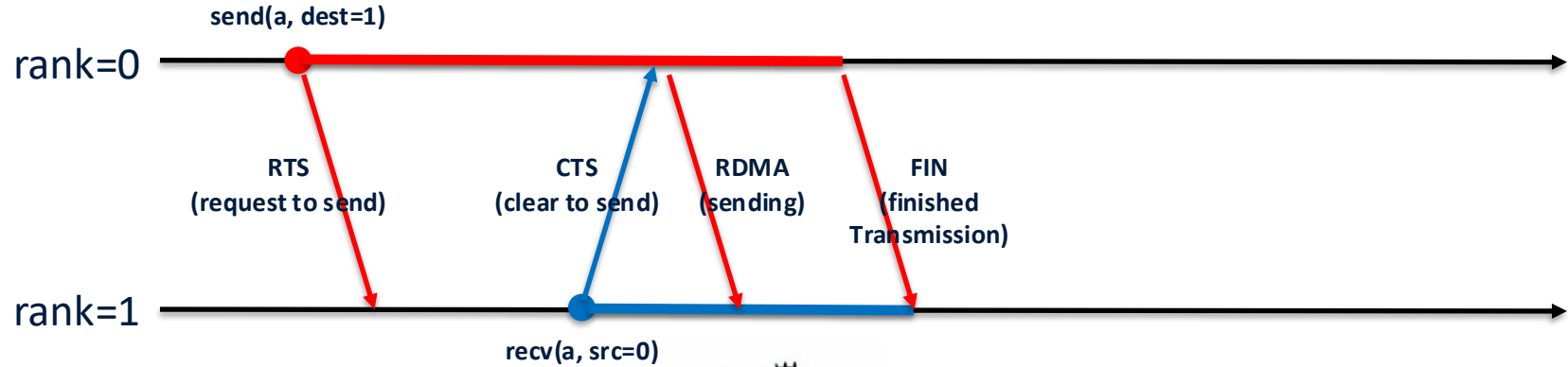
02-MPI-introduction.ipynb

- Step-by-step introduction to MPI concepts in Python (mpi4py).

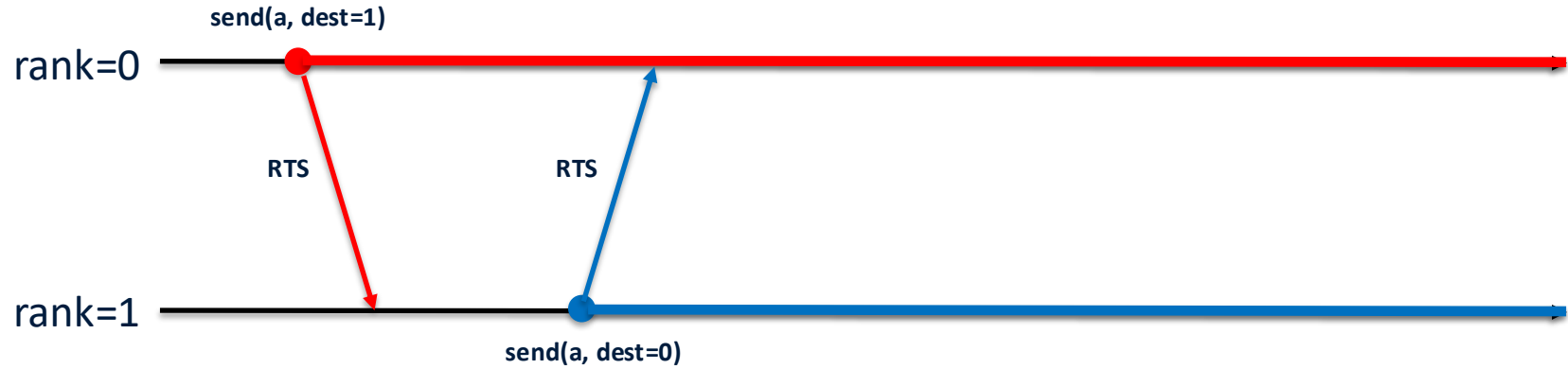
03-domain-decomposition.ipynb

- Learn about domain-decomposition.
- Apply domain-decomposition to a simple 1d example.
- Apply domain-decomposition to the stencil2d.py program.

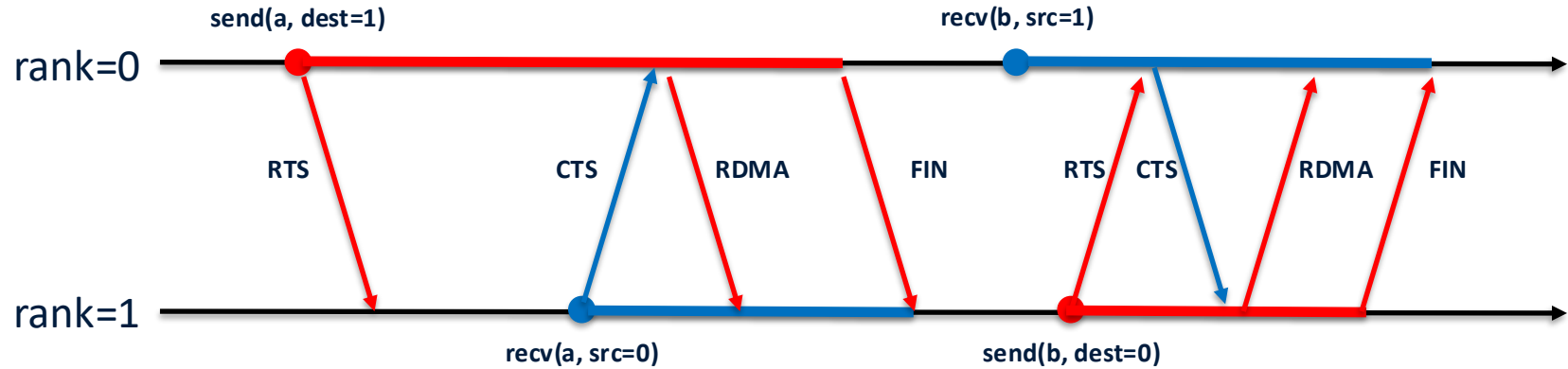
Send / Receive (Rendezvous protocol = large messages)



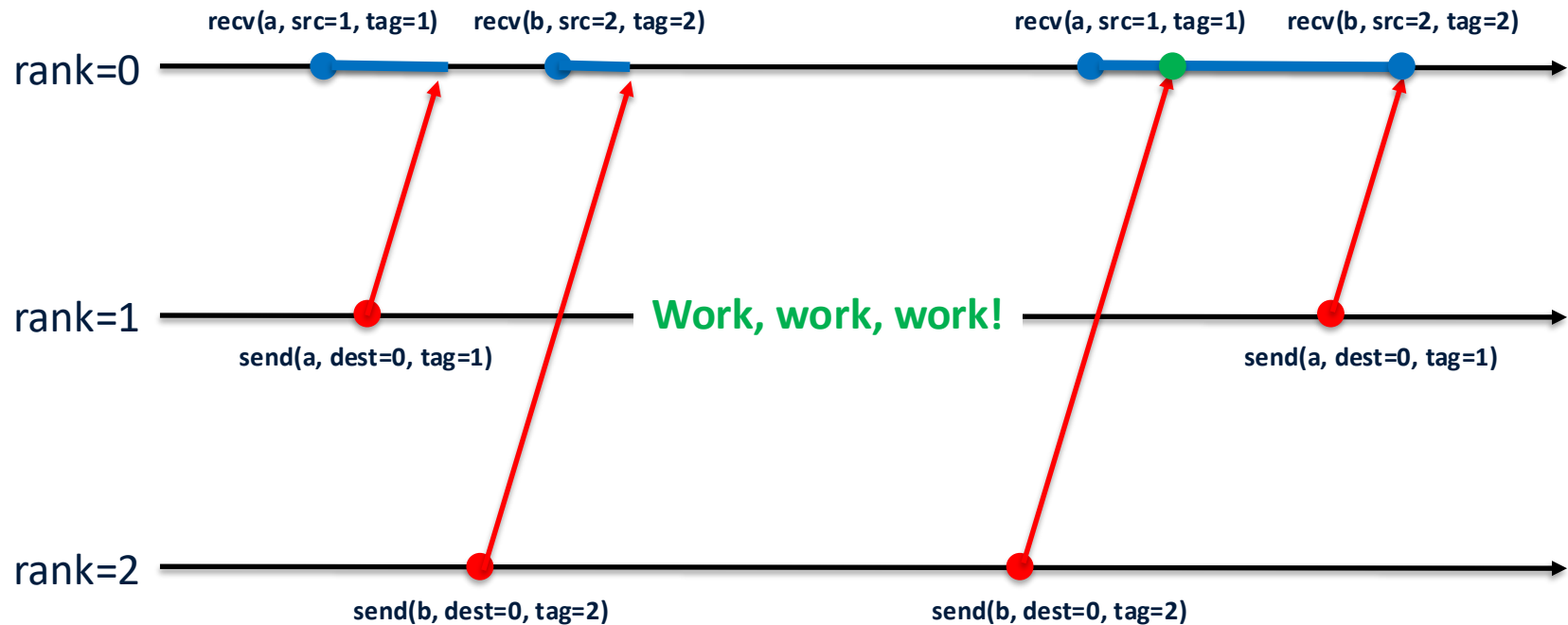
Deadlock



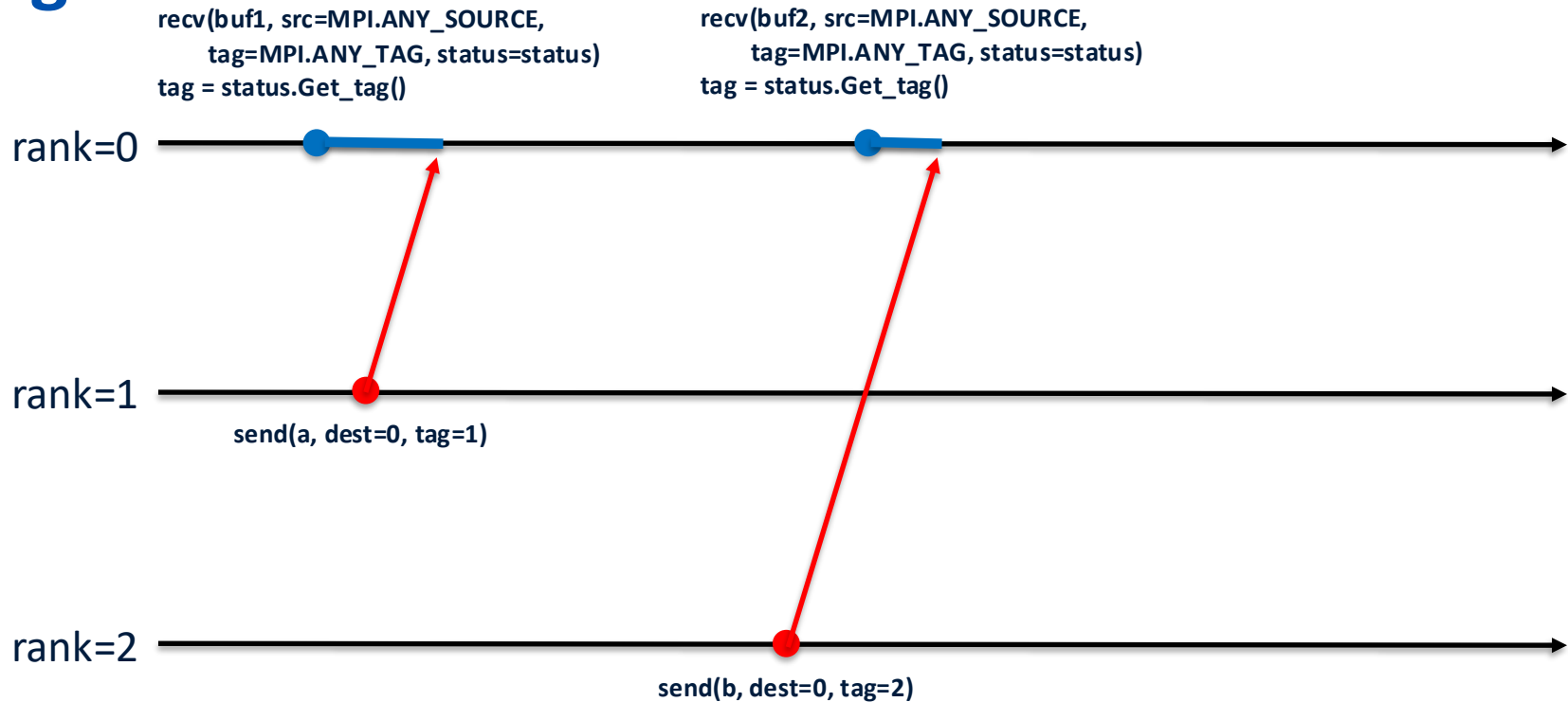
Matching Send / Recv



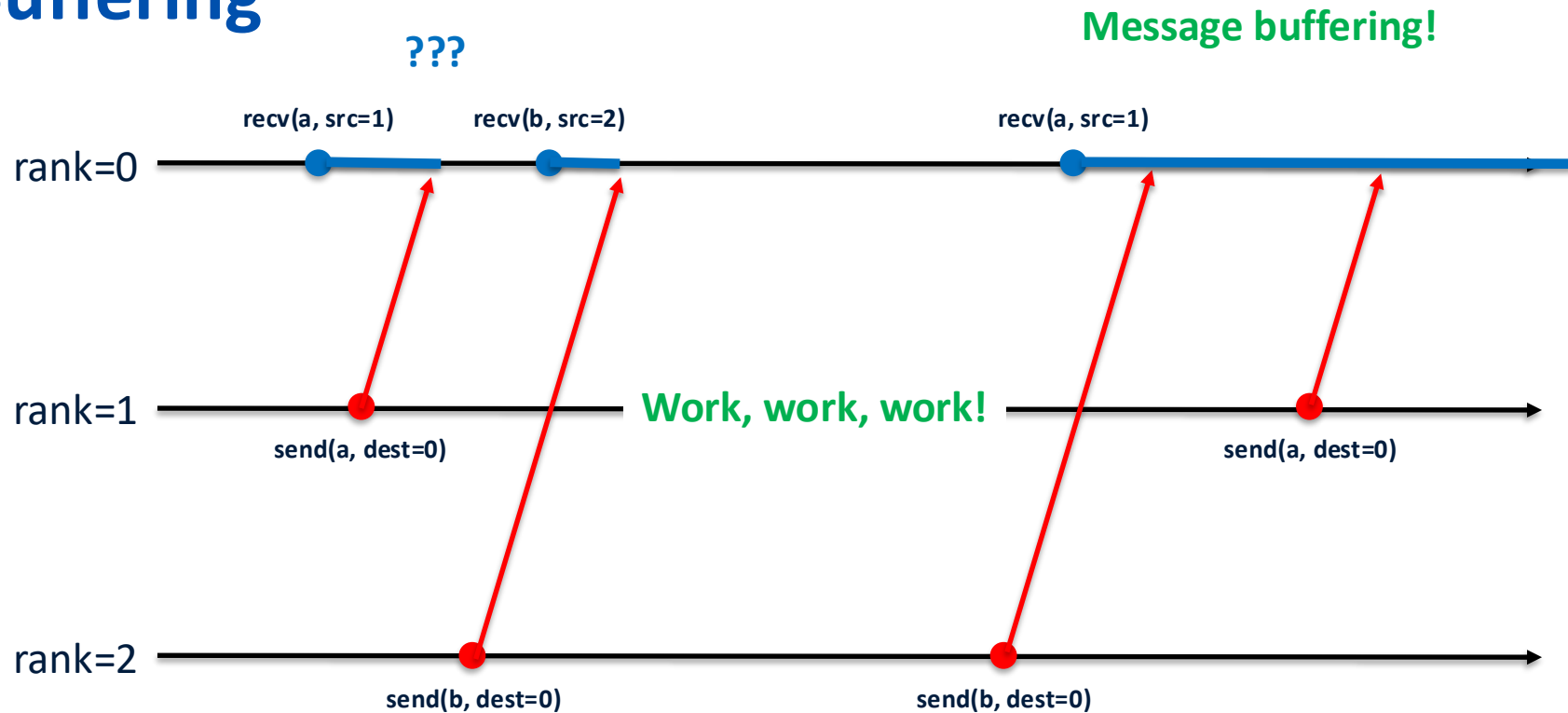
Tags



Tags



Buffering



Rules

The order of arrival between two messages from the same sender to the same receiver is preserved by MPI

Each Send/Irecv must match
a Recv/Isend (source, tag, communicator)

Use tags to disambiguate messages,
especially when order differs

Blocking communication

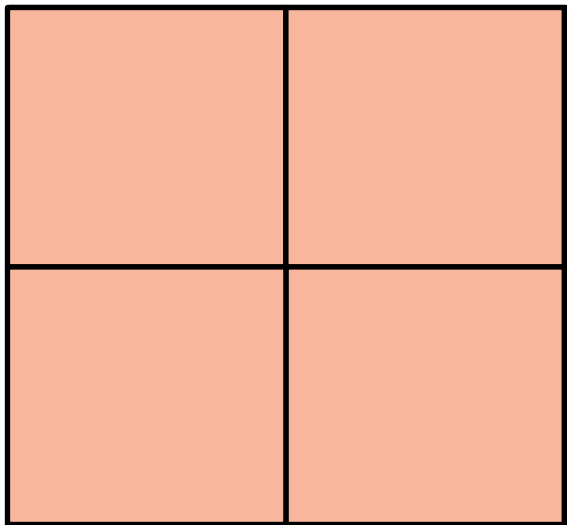
- Send and Recv must occur in the same order, or deadlocks may occur
- Tags alone do not resolve ordering or prevent deadlock
- Blocking calls complete before control returns

Non-blocking communication

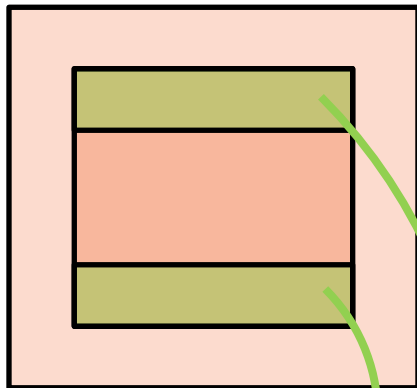
- Order of Isend/Irecv is more flexible (but tags are highly recommended)
- Every non-blocking op must be followed by Wait, Waitall, Test, etc.
- Never modify send/receive buffers until the operation completes

Corners

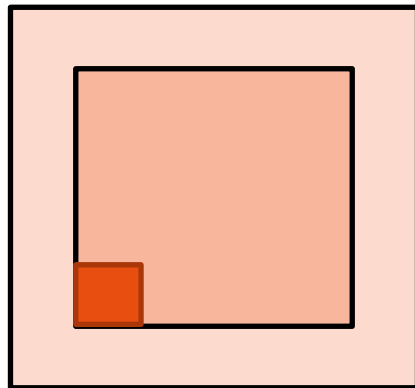
global domain



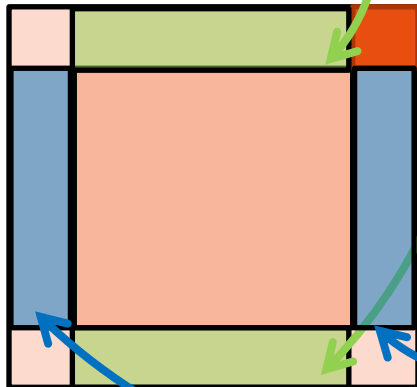
rank = 2



rank = 3

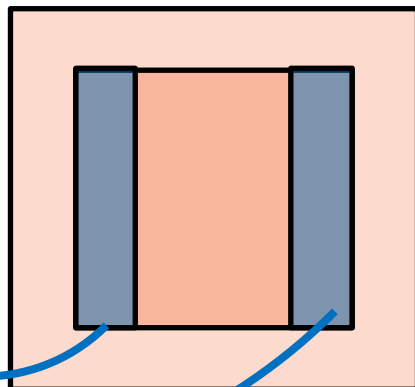


rank = 0



rank = 1

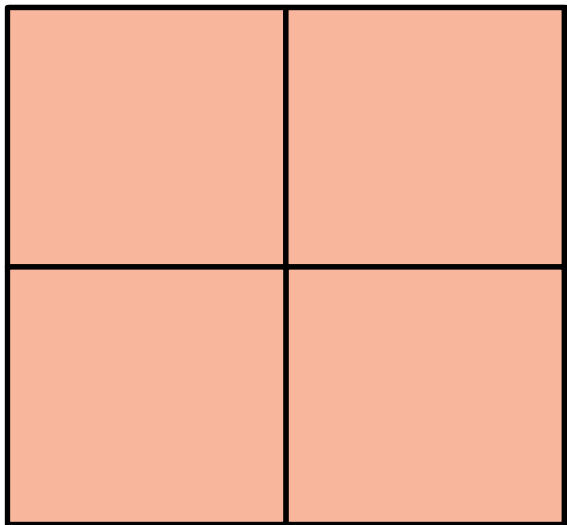
top-bottom



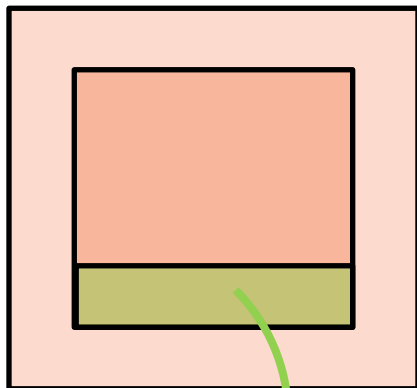
left-right

Corner Strategy 1

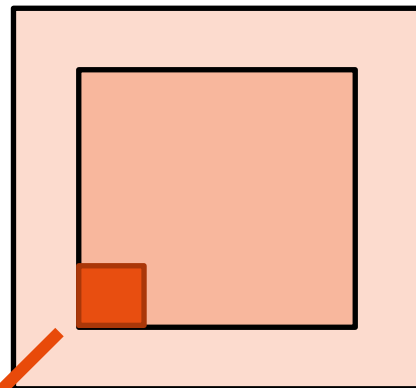
global domain



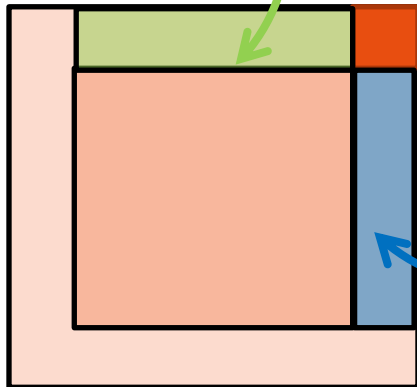
rank = 2



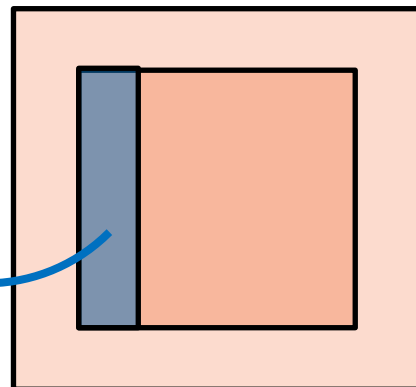
rank = 3



rank = 0

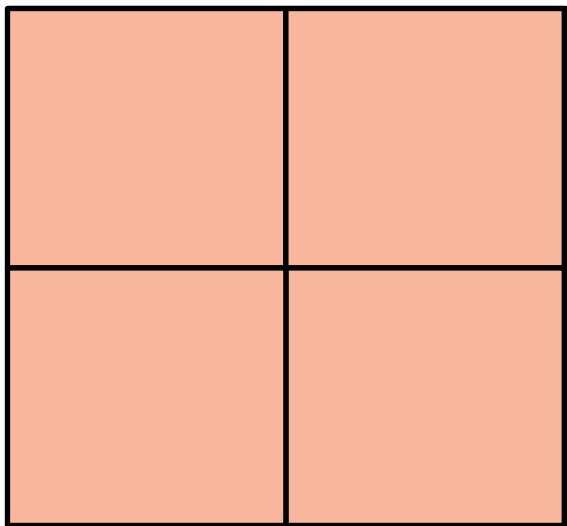


rank = 1

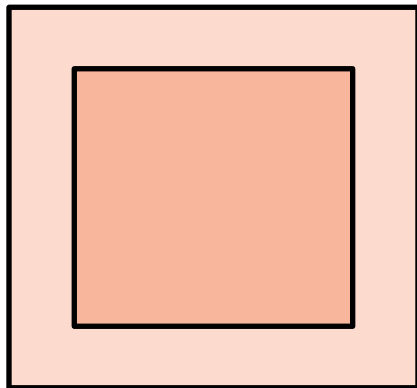


Corner Strategy 2

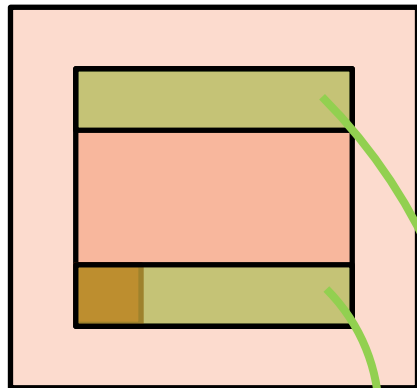
global domain



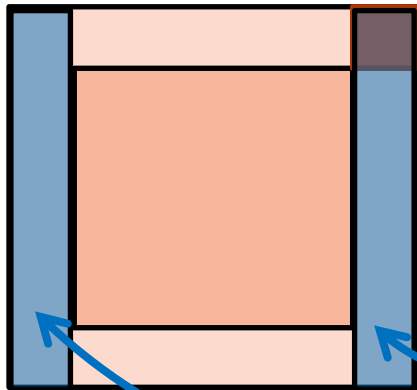
rank = 2



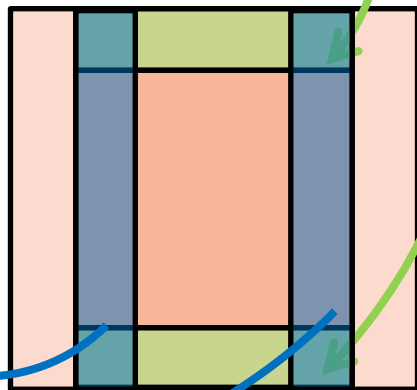
rank = 3



rank = 0



rank = 1

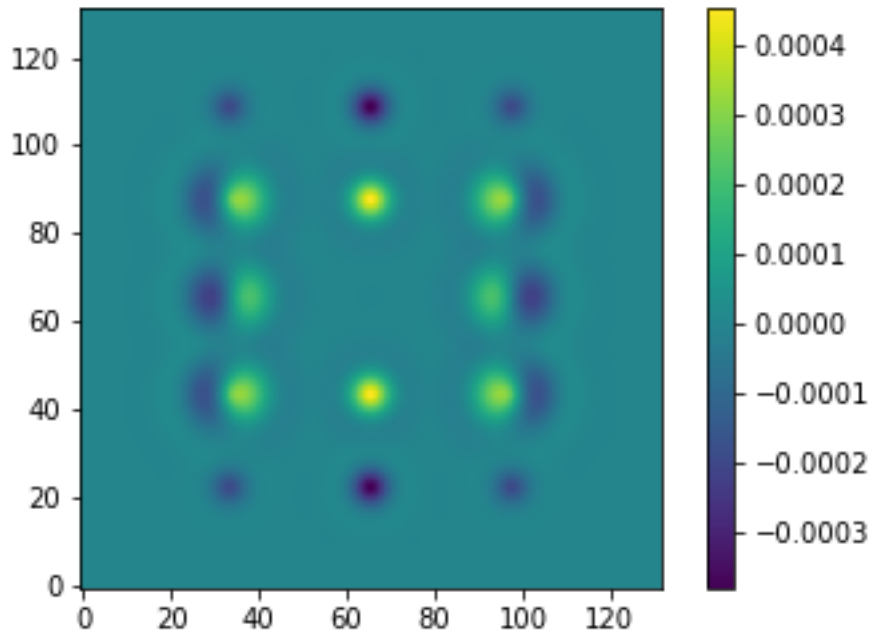
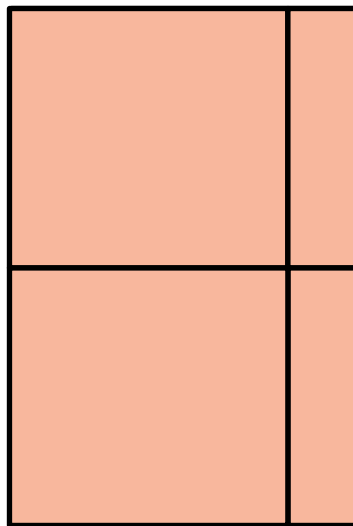


top-bottom

left-right

Error without sync

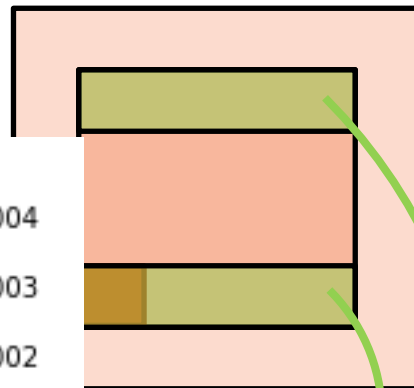
global domain



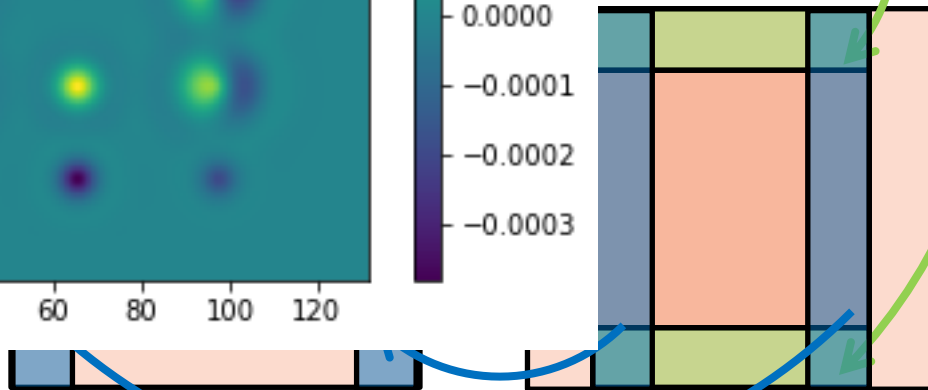
rank = 2



rank = 3



$k = 1$ top-bottom



left-right